Veloce Logic – Penetration Test Report

Oct 14, 2026 · @Gopal

Project & Group Information

Enterprise Name: Veloce Logic

Assigned Project Title: Threat Modeling and Containment of Content Management System Exploitation in Staging Nodes

Target Lab Environment: TryHackMe — Boiler CTF (room link)

Group Members:

#	Student Name	TryHackMe Profile Link	Kali Username
1	Gopal Kovachi	kovachigopal1	gopal
2	Gulshan Pandit	gulshanp2569	Gulshan
3	Hasan Ali	hxnn6	hasan
4	Kanha Patel	kanhaPatel	kanhapatel

Technical PoC Archive (Google Sheet): Veloce-Logic-Pentest-PoC

Executive Summary

Overview. A penetration test was performed against a single host (10.49.174.61) hosting a Joomla-based website, codenamed "Boiler CTF." The objective was to determine whether an external attacker, with no prior credentials or access, could compromise the server. Full coverage of the host was achieved.

Results. The assessment found that the host could be fully compromised, from an anonymous, unauthenticated starting point through to complete administrative (root) control of the underlying server. The test did not stop at a single flaw: a chain of smaller issues, none requiring any special access, combined to produce total compromise.

Impact. Had this host been in production, an attacker on the internet could have read and modified any file on the server, installed persistent malware, used the server to pivot into the internal network, and exposed any sensitive data it held. No specialist skills or insider knowledge were required; publicly known attack techniques and tools were sufficient.

Remediation Direction. The most urgent action is to remove or patch the vulnerable third-party tool that allowed remote command execution (see Findings & Recommendations). This alone would have blocked the entire attack chain. Beyond that, the organisation should review how credentials are stored and logged, and correct file permissions that allowed a standard user to gain full administrative rights. None of the required fixes represent major investment; they are configuration and patching changes that can be completed quickly.

Findings & Recommendations

Five findings were raised. Each is rated in isolation, as if the others did not exist, but the true risk comes from how they chain together (see below).

#	Finding	Risk Rating
1	Anonymous FTP Access Enabled	Low
2	Unauthenticated OS Command Injection in sar2html	Critical
3	Plaintext Credential Disclosure via Readable Log File	High
4	Hardcoded Credentials in Backup Shell Script	High
5	Privilege Escalation via Misconfigured SUID Binary (find)	Critical

Attack chain. While Finding 1 (anonymous FTP) is low risk on its own, Findings 2 through 5 form a direct path from no access to root: the command injection (2) was used to read a log file never meant to be web-accessible (3), which disclosed a password for a low-privileged account. That account's home directory held a backup script with a second, reused-pattern password for another account (4). That second account could run the `find` command as root due to a misconfigured SUID permission, giving full read/write access to the entire filesystem (5). This is a systemic problem: no single team caused it, but secure configuration review was missing at the web application layer, the logging layer, the credential-management layer, and the operating-system hardening layer. Addressing

Finding 2 alone would have broken this chain at its first link and is the single highest-priority fix.

Finding 1: Anonymous FTP Access Enabled

Risk Rating: Low (CVSS 3.1 Base Score: 5.3)

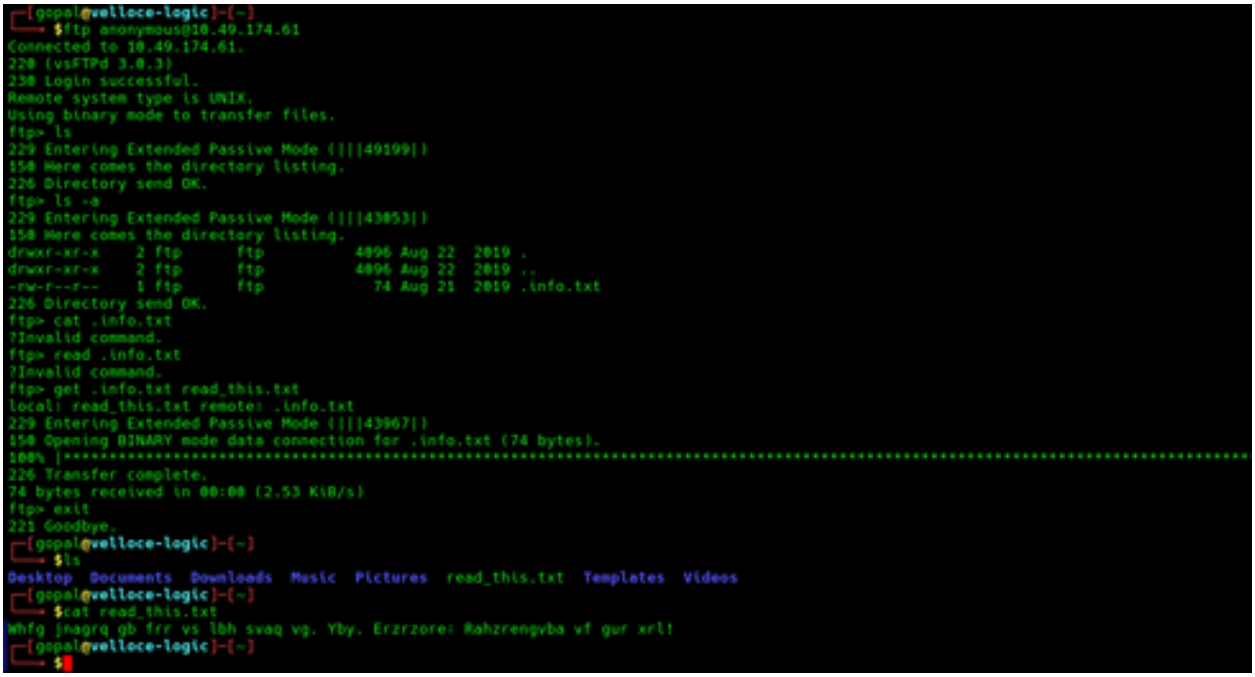
Summary: The FTP service on port 21 accepted anonymous logins, allowing any unauthenticated user to browse and download files from the FTP root.

Background: vsFTPd, the FTP daemon in use, supports an "anonymous" account that requires no password by convention. When left enabled on an internet-facing server, this allows any visitor to list and retrieve files without any credentials, which can expose files never intended for public access.

Technical Details & Evidence: An Nmap scan (`nmap -sCV -p- 10.49.174.61`) identified vsFTPd 3.0.3 on port 21 and flagged anonymous login as allowed. This was confirmed by connecting directly:

```
ftp anonymous@10.49.174.61
230 Login successful.
```

A directory listing revealed a single file, `.info.txt`, which was downloaded and reviewed.



Impact: In this instance, the exposed file contained only a message left by the system owner and did not disclose sensitive data. However, anonymous FTP is a common vector

for exposing configuration files, credentials, or source code, and should be treated as a finding regardless of what the current contents happen to be.

Remediation Advice: Disable anonymous FTP access in `/etc/vsftpd.conf` by setting `anonymous_enable=NO` and restarting the service. If public file distribution is genuinely required, serve those files over HTTPS from a dedicated, access-controlled directory instead of FTP.

References:

- https://security.appspot.com/vsftpd/vsftpd_conf.html

Finding 2: Unauthenticated OS Command Injection in sar2html

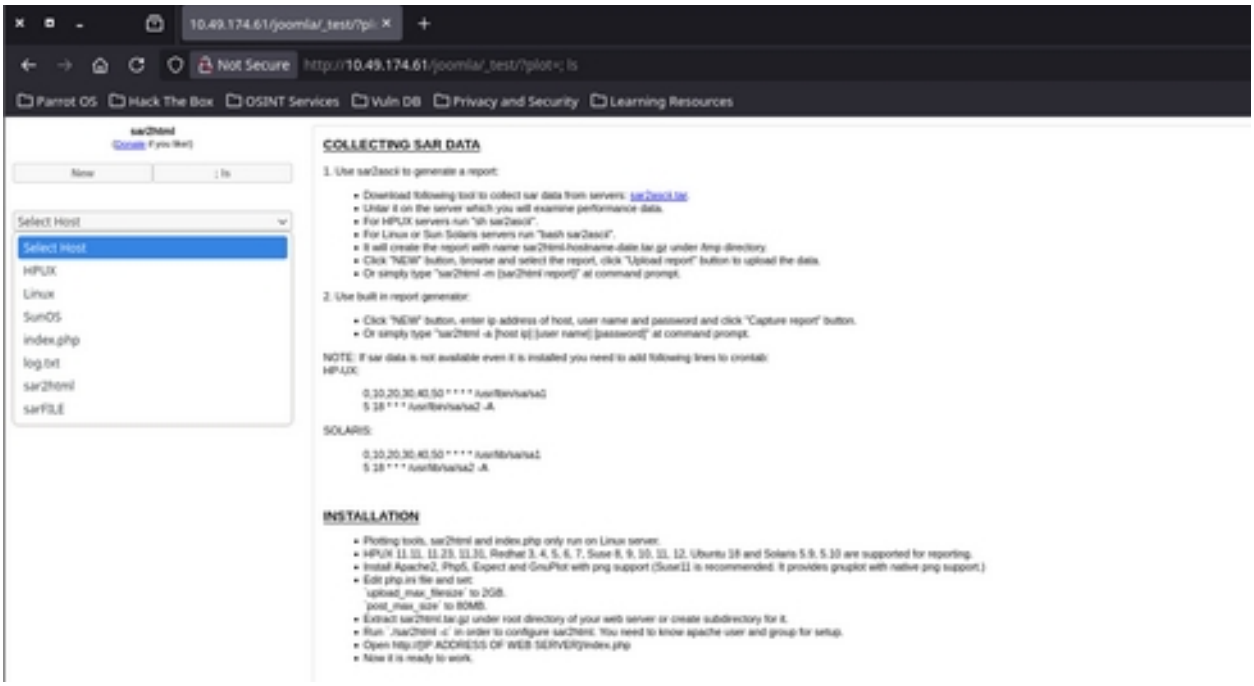
Risk Rating: Critical (CVSS 3.1 Base Score: 9.8)

Summary: A sar2html instance hosted at `/joomla/_test/` on the web server was vulnerable to unauthenticated OS command injection via its `plot` parameter, allowing arbitrary commands to run as the web server user with no login required.

Background: sar2html is a third-party tool for rendering `sar` performance logs as HTML. Public versions up to 3.2.2 pass the `plot` query parameter directly into a shell command without sanitisation (tracked publicly as CVE-2020-36604). Any character that has meaning to the shell, such as a semicolon, lets an attacker append their own command to the one the application intended to run.

Technical Details & Evidence: Directory brute-forcing with Gobuster and Dirb against `/joomla/` (wordlist: `directory-list-2.3-medium.txt` / `common.txt`) identified a `/joomla/_test/` directory running sar2html. Appending a semicolon and a command to the `plot` parameter executed that command and returned its output in the page:

```
GET /joomla/_test/?plot=;ls
GET /joomla/_test/?plot=;cat log.txt
```



The first request confirmed arbitrary command execution by listing the working directory (returning `log.txt`, `sar2html`, `sarFILE`, among others). The second request read the contents of `log.txt` directly from the browser.

Impact: This vulnerability alone grants an unauthenticated internet-based attacker full command execution as the web server account. In this engagement it was used to read a log file containing a plaintext credential (Finding 3), which was the pivot point for full server compromise (Findings 4–5). Beyond file reads, the same injection point could be used to download and run a reverse shell, achieving fully interactive remote access.

Remediation Advice: Remove the sar2html application from the production web server; it appears to be a testing/monitoring tool that was never intended for public internet exposure. If sar2html functionality is required, upgrade to a patched release that validates and escapes the `plot` parameter before passing it to a shell, and place the tool behind authentication and network-level access control (e.g. restricted to an internal management VLAN) as defence in depth. Input validation alone is not sufficient; the root cause is unsanitised input reaching a shell command, which must be fixed at the code level or by replacing the vulnerable component.

Finding 3: Plaintext Credential Disclosure via Readable Log File

Risk Rating: High (CVSS 3.1 Base Score: 7.5)

Summary: A log file (`log.txt`) in the sar2html working directory recorded an SSH login event that included the account's password in plaintext. Because the file was readable by the web application account, it was exposed by Finding 2.

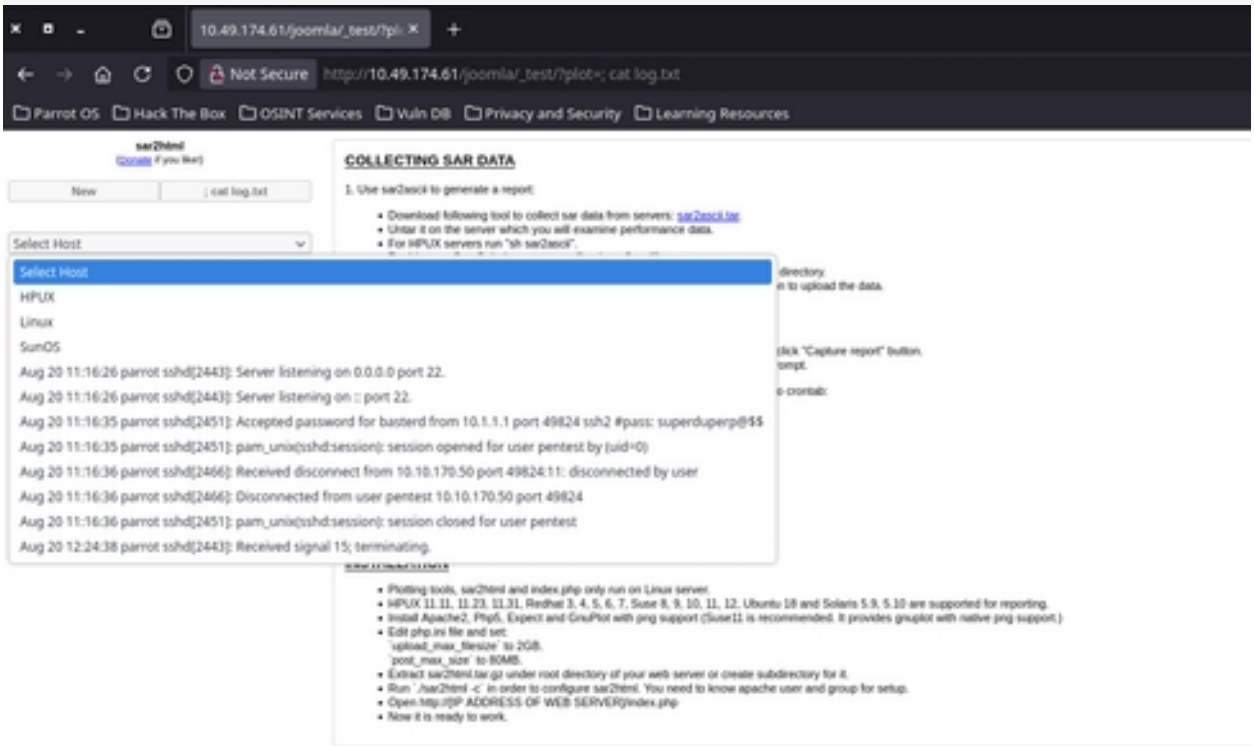
Background: Authentication logs should record that a login occurred, by whom, and when, but should never include the credential itself. A custom logging or wrapper script on this host appears to append the password used for an SSH session to the log entry (visible as `#pass: <password>`), which is not standard `sshd` behaviour and indicates a local logging customisation.

Technical Details & Evidence: Using the command execution obtained in Finding 2, the log file was read:

```
GET /joomla/_test/?plot=;cat log.txt
```

The output included the line:

```
Aug 20 11:16:35 parrot sshd[2451]: Accepted password for basterd from 10.1.1.1
port 49824 ssh2 #pass: superduperp@$$
```



This disclosed valid SSH credentials for the local account `basterd`.

Impact: The disclosed credential was used to authenticate over SSH as `basterd` on port 55007, providing the first interactive shell on the host and the foothold needed for the privilege escalation chain in Findings 4 and 5. Any file reachable by the web server account that contains credentials represents a direct path from a web-layer compromise to full account takeover.

Remediation Advice: Identify and remove the mechanism appending plaintext passwords to `log.txt`; standard `sshd` logging does not do this, so it is most likely a custom script or PAM module and should be located and disabled. Rotate the credential for the `basterd` account immediately, as it must be considered compromised. Going forward, ensure log files that may contain sensitive data are stored outside any web-accessible directory and are not readable by the web server's service account.

Finding 4: Hardcoded Credentials in Backup Shell Script

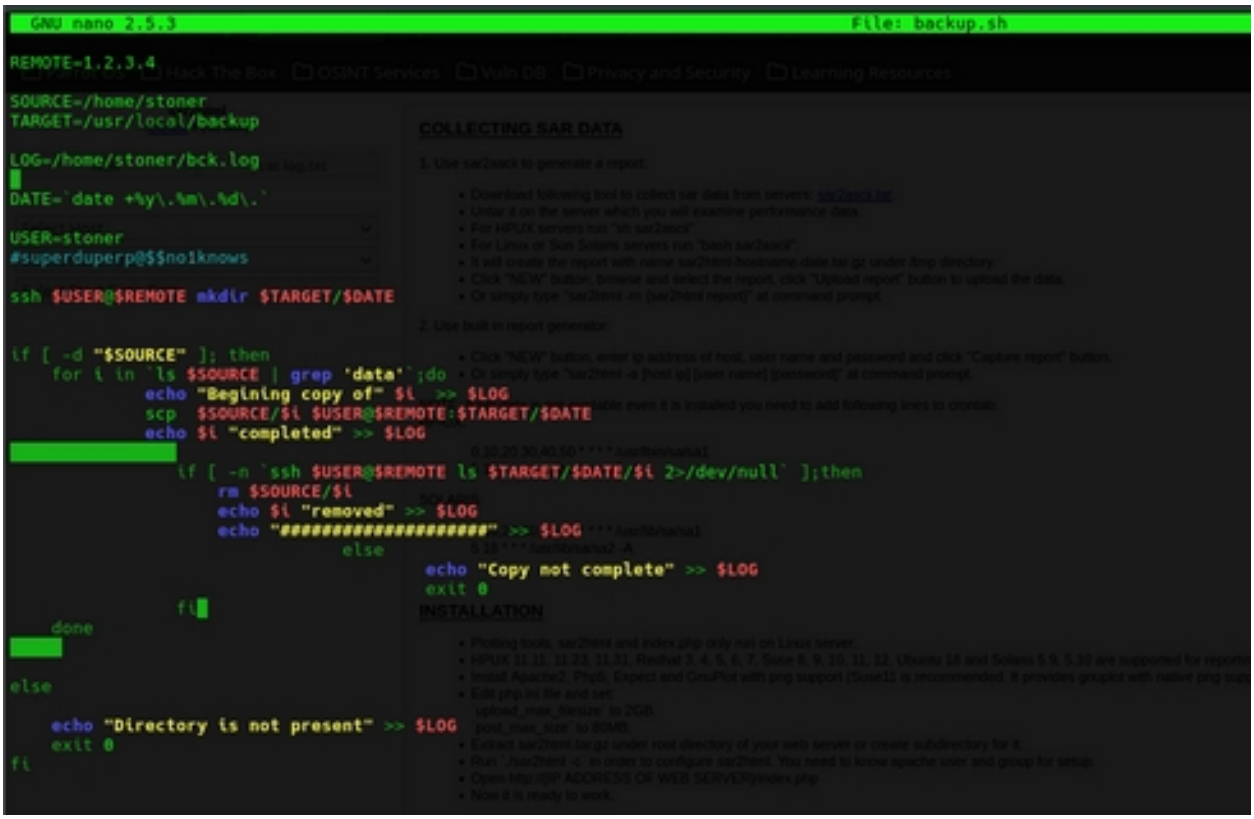
Risk Rating: High (CVSS 3.1 Base Score: 8.1)

Summary: A shell script (`backup.sh`), accessible from the `basterd` account obtained in Finding 3, contained a second account's password in a comment, enabling horizontal movement to the `stoner` account.

Background: Scripts that automate tasks such as backups often need credentials to run unattended. Storing those credentials directly in the script, even as a comment rather than an active variable, still leaves them in plaintext on disk for anyone who can read the file. A comment is not a security control; it is simply text that a shell interpreter ignores but a person reading the file will not.

Technical Details & Evidence: After authenticating as `basterd`, `backup.sh` was reviewed. The script defines a backup routine (`USER=stoner`, copying files to a `$REMOTE` host via `scp`) and includes a commented-out line directly beneath the `USER` variable:

```
USER=stoner
#superduperp@$$no1knows
```



The string follows the same base phrase as the password disclosed in Finding 3, with an appended suffix, indicating password reuse/pattern reuse across accounts. This value was used to authenticate over SSH as `stoner`.

Impact: This finding allowed movement from the `basterd` account to the `stoner` account. It also demonstrates a broader password-reuse pattern across the host's local accounts, meaning the compromise of one credential materially increases the risk of every other account rather than being contained to a single user.

Remediation Advice: Remove all credentials from `backup.sh` and any similar scripts. For automated tasks such as this backup job, use SSH key-based authentication scoped to the specific task, or a secrets manager/vault that injects credentials at runtime rather than storing them on disk. Separately, enforce a password policy that prevents reuse of the same base password (with or without suffixes) across multiple accounts, and rotate the `stoner` credential.

Finding 5: Privilege Escalation via Misconfigured SUID Binary (/usr/bin/find)

Risk Rating: Critical (CVSS 3.1 Base Score: 9.1)

Summary: The `find` binary had the SUID permission bit set and was owned by `root`. Because `find` has a documented `-exec` option, any local user could use it to run arbitrary commands as `root`, giving full administrative control of the host.

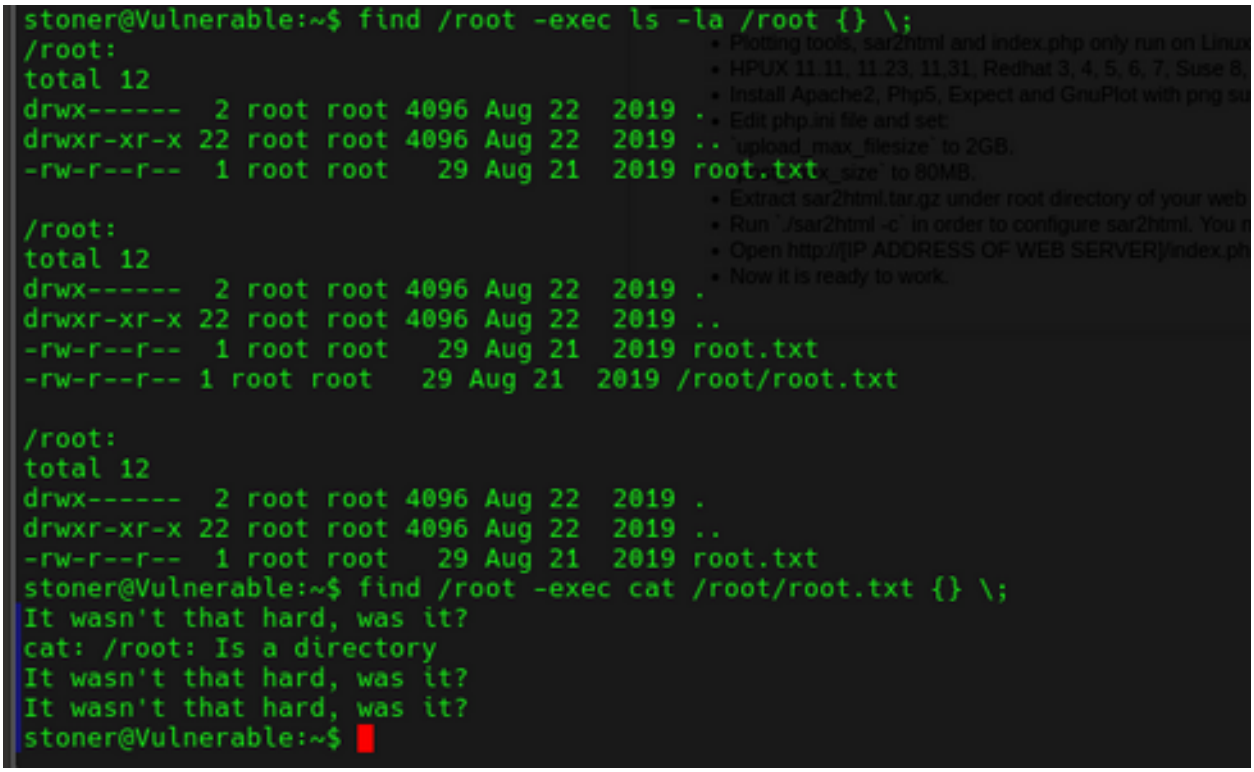
Background: The SUID (Set User ID) bit causes a program to run with the permissions of its file owner rather than the user who launched it. This is appropriate for a small number of system utilities that genuinely need elevated access for one specific task (such as `passwd`), but is dangerous on general-purpose utilities that can be made to execute other commands on the owner's behalf. `find` is one of several common binaries tracked by the public GTFOBins project precisely because its `-exec` flag can be abused this way whenever SUID is set.

Technical Details & Evidence: After authenticating as `stoner` (Finding 4), a search for SUID binaries owned by root was run:

```
find / -user root -perm -4000 -exec ls -ldb {} \;
```

This returned `/usr/bin/find` among the results, with owner `root` and the SUID bit set. This was then used to read a file normally restricted to `root`:

```
stoner@Vulnerable:~$ find /root -exec cat /root/root.txt {} \;
It wasn't that hard, was it?
```



The command successfully read `/root/root.txt` as the `root` user from the unprivileged `stoner` session, confirming arbitrary file read as root. The same technique (`find . -exec /bin/sh -p \;`) would equally provide an interactive root shell.

Impact: This is a complete compromise of the host. With root-equivalent command execution, an attacker can read or modify any file, create new privileged accounts, install persistent backdoors, disable logging, and use the host as a pivot point into any network it can reach.

Remediation Advice: Remove the SUID bit from `/usr/bin/find` (`chmod u-s /usr/bin/find`); this binary does not require it for normal operation. More broadly, audit all SUID/SGID binaries on the host against the standard set shipped by the OS distribution (`find / -perm -4000 -o -perm -2000`) and remove the bit from any that are not required, checking the GTFOBins list for binaries known to be abusable when SUID is set. This audit should be a standard step in server hardening, not a one-off fix.

Cyber Kill Chain Mapping

The attack path executed during this engagement is mapped below to Lockheed Martin's Cyber Kill Chain. Full supporting evidence (commands, screenshots, raw output) is in the Technical PoC Archive: Veloce-Logic-Pentest-PoC

Kill Chain Phase	Activity Performed	Related Finding
1. Reconnaissance	Port/service scan (`nmap -sCV -p-`) identified FTP, HTTP (Joomla), Webmin and SSH; directory brute-forcing (Gobuster, Dirb) mapped the Joomla install and located a hidden `/joomla/_test/` directory.	Findings 1, 2
2. Weaponization	No custom payload was built; the attack weaponized a known, unpatched flaw already present in the target's software (unsanitised `plot` parameter in the exposed sar2html instance).	Finding 2
3. Delivery	The malicious "payload" was delivered as a crafted HTTP GET request to the vulnerable endpoint — no email, USB or social-engineering vector was required, as the flaw was directly internet-reachable.	Finding 2
4. Exploitation	The `plot` parameter's lack of input sanitisation was triggered, causing the application to pass attacker-supplied shell metacharacters straight to the system shell (unauthenticated OS command injection).	Finding 2
5. Installation	No malware or web shell was installed; the command-injection point	Finding 2

Kill Chain Phase	Activity Performed	Related Finding
	itself served as a standing, repeatable execution primitive for as long as the vulnerable application remained deployed.	
6. Command & Control	Command execution was driven directly over HTTP(S) through the injection point (no outbound C2 channel/beacon was needed), and was later upgraded to an interactive SSH session using credentials harvested through that access.	Findings 2, 3, 4
7. Actions on Objectives	Credentials disclosed via the injection point were reused to pivot between two local accounts, and a misconfigured SUID binary was then abused to obtain root-equivalent command execution — full compromise of the host.	Findings 3, 4, 5

Defensive Remediation & Hardening Plan

Code Fixes: Eliminating the Command Injection (Finding 2)

The root cause of Finding 2 was untrusted input (the `plot` parameter) being concatenated into a shell command. The fix is to never pass user input to a shell, and instead call the intended program directly with an argument array, which is not interpreted by a shell:

```
// VULNERABLE — user input concatenated into a shell command string
$cmd = "sar2html -f " . $_GET['plot'];
system($cmd);

// FIXED — allow-list the expected values; never build a shell string from
```

```
input
$allowed = ['HPUX', 'Linux', 'SunOS', 'index.php', 'log.txt', 'sar2html',
'sarFILE'];
if (!in_array($_GET['plot'], $allowed, true)) {
    http_response_code(400);
    exit('Invalid selection.');
}
// If a subprocess is unavoidable, use escapeshellarg() AND an argument array
// (never string concatenation), e.g. proc_open() with a args array, not
system().
```

This same pattern (reject-by-default, allow-list known-good values, never build a command string from request data) should be applied to every parameter across the application that reaches a shell, a file path, or a database query, not just this one endpoint.

Server Hardening: Linux Configuration Rules

Area	Rule
Vulnerable software	Remove the sar2html application from the production web root, or upgrade to a patched release and move it behind authentication on an internal-only network segment.
Upload/execution directories	Mount web-writable directories `noexec` where possible, and ensure Apache is configured to deny script execution outside the application's own code directories (`Options -ExecCGI`, no `.php`/`.cgi` handlers mapped onto uploads or log paths).
Anonymous FTP	Set `anonymous_enable=NO` in `/etc/vsftpd.conf` and restart the service (Finding 1).
Log file exposure	Store application/service logs outside any web-accessible directory, and set restrictive permissions (`chmod 640`, owned by a dedicated logging user) so the web server account cannot read them (Finding 3).
Credential logging	Locate and remove the custom logging

Area	Rule
	hook that appends plaintext passwords to `log.txt`; SSH logging should never include the credential itself (Finding 3).
Hardcoded credentials	Remove plaintext/commented credentials from scripts such as `backup.sh`; use SSH key pairs scoped to the task or a secrets manager instead (Finding 4).
Password reuse	Enforce a password policy (e.g. via `pam_pwquality`/`pam_cracklib` plus organisational process) that blocks the same base password being reused across local accounts (Finding 4).
SUID/SGID binaries	Remove unnecessary SUID bits: `chmod u-s /usr/bin/find`. Run `find / -perm -4000 -o -perm -2000 -type f 2>/dev/null` on a schedule and diff the output against a known-good baseline, checking any new or unexpected entries against GTFOBins (Finding 5).
General	Apply the principle of least privilege throughout: service accounts should run with the minimum permissions needed, and any one compromised credential or component should not be enough, on its own, to reach root.

Appendix A: Assessment Scope

Target: 10.49.174.61 (single host, codenamed "Boiler CTF")

Testing window: 14 October 2026

Services identified (via `nmap -sCV -p-`):

Port	Service	Version	Tested
21/tcp	FTP	vsFTPd 3.0.3	Yes — Finding 1
80/tcp	HTTP	Apache 2.4.18	Yes — Findings 2–5

Port	Service	Version	Tested
		(Ubuntu), hosting Joomla	reached via this service
10000/tcp	HTTP (Webmin)	MiniServ 1.930	Identified, not exploited
55007/tcp	SSH	OpenSSH 7.2p2 Ubuntu	Yes — used post-credential-disclosure

Coverage achieved: Full compromise of the host was achieved, from unauthenticated access through to root-equivalent command execution, which represents complete coverage of the realistic attack surface for this engagement. Directory enumeration (Gobuster, Dirb) additionally identified `/joomla/_archive/`, `/joomla/_database/`, and `/joomla/_files/`; these were reviewed but did not contain usable data — in particular, SQL files located under the `tests` index were present but empty.

Out of scope / not tested: The Webmin service on port 10000 was identified but not investigated further, as full compromise was already achieved via the web application and SSH vector. It is recommended that this service be assessed in a future engagement, given Webmin's own history of publicly disclosed vulnerabilities.

Appendix B: Assessment Artefacts

This section lists anything the test left on the target, so it is not later mistaken for evidence of a real attack.

Files created or modified on the target: None. All actions performed through the command injection (Finding 2) and on the host directly (Findings 4–5) were read-only reconnaissance commands (`ls`, `cat`, `find ... -exec ls`, `find ... -exec cat`). No files were uploaded, no new user accounts were created, and no persistence mechanisms were installed.

Interactive sessions: Two SSH sessions were established and will appear in the host's authentication logs:

Account	Port	Purpose
`basterd`	55007	Initial foothold using the credential disclosed in Finding 3
`stoner`	55007	Lateral movement using

Account	Port	Purpose
		the credential disclosed in Finding 4

The client should review `auth.log` / `wtmp` around the testing window for these two logins alongside the command-injection requests to `/joomla/_test/?plot=...` in the Apache access log, to confirm the activity matches this report and to distinguish it from any unrelated access.

Credentials used: Both credentials above were discovered during testing rather than supplied in advance, and both accounts should be treated as compromised and rotated regardless of this being a test engagement, since the same credentials were readable by any party who reached the same files.

Notes: Decoy Content

For completeness, the following items were investigated during testing but determined to be intentional red herrings placed by the lab's author rather than genuine vulnerabilities. They are documented here so they are not mistaken for missed findings:

- **`.info.txt` (FTP root):** Contained a ROT13-encoded string. Decoded, it read: "Just wanted to see if you find it. Lol. Remember: Enumeration is the key!"
- **`robots.txt`:** Listed several plausible-looking disallowed paths, followed by a string of decimal values. Converting the values to ASCII characters produced a Base64 string, which decoded to an MD5 hash (`99b0660cd95adea327c54182baa51584`); cracking the hash returned the plaintext "kidding".
- **`.secret` (in `stoner`'s home directory):** Contained the message "You made it till here, well done."

None of these items granted access, disclosed credentials, or otherwise contributed to the compromise chain described in this report.